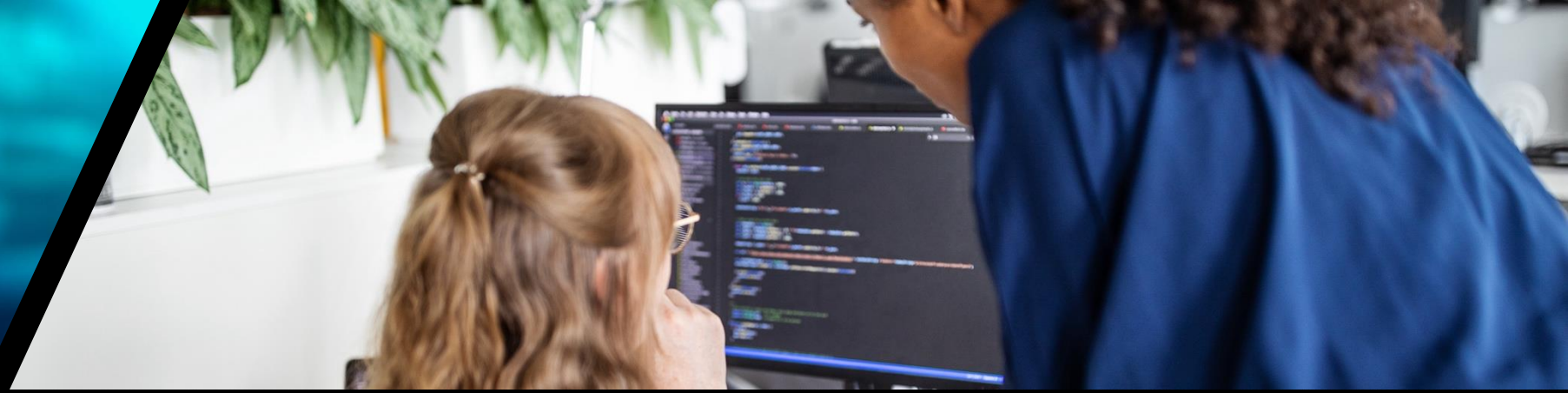
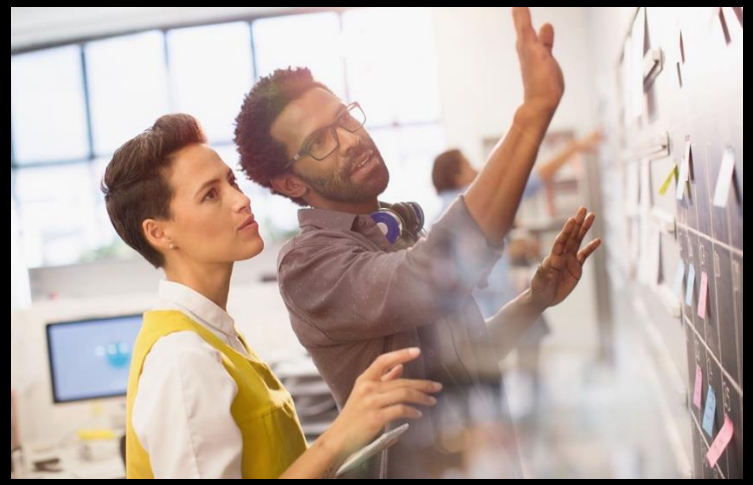


A tour of software design



CSC207 Software Design

An introduction to software design and development concepts, methods, and tools using a statically-typed object-oriented programming language such as Java. Topics from: version control, unit testing, refactoring, object-oriented design and development, design patterns, advanced IDE usage, regular expressions, and reflection.

Agenda

Hour 1

- Introductions (activity!)
- Big Picture of the Course
- Team-Based Learning (activity!)

Hour 2

- Intro to our tools (git, Java, IntelliJ)
- Demo in IntelliJ

Menti for Q+A

https://www.menti.com/alvfjy54rw1n

1990 8756

Introductions

Me (Jonathan)

Lindsey

Paul

Activity

Introduce yourself to your neighbour(s):

- what is your name?
- what is your PoSt?
- what is a programming project or assignment that you found interesting?

Setting the course context

Programming teams grow large, complex applications

- Where do application ideas come from?
- What other roles are involved in designing and creating an application?
- How do programmers discuss the code and what abstractions do they use?
- What tools do they use?
- How do they distribute the work?
- How do they share the code?

Course Material and Tools

Modules

M0 – Java Fundamentals and Version Control

M1 – Java OOP

M2 – Design Principles

M3 – Clean Architecture

M4 – Design Patterns

M5 – Ethics, Regex, and Presentations

Tools

- Java
- IntelliJ
- Git
- Design Principles and Patterns (things to do)
- Code Smells (things to avoid)
- An architecture
- APIs

Course Material and Tools

Modules

M0 – Java Fundamentals and Version Control

M1 – Java OOP

M2 – Design Principles

M3 – Clean Architecture

M4 – Design Patterns

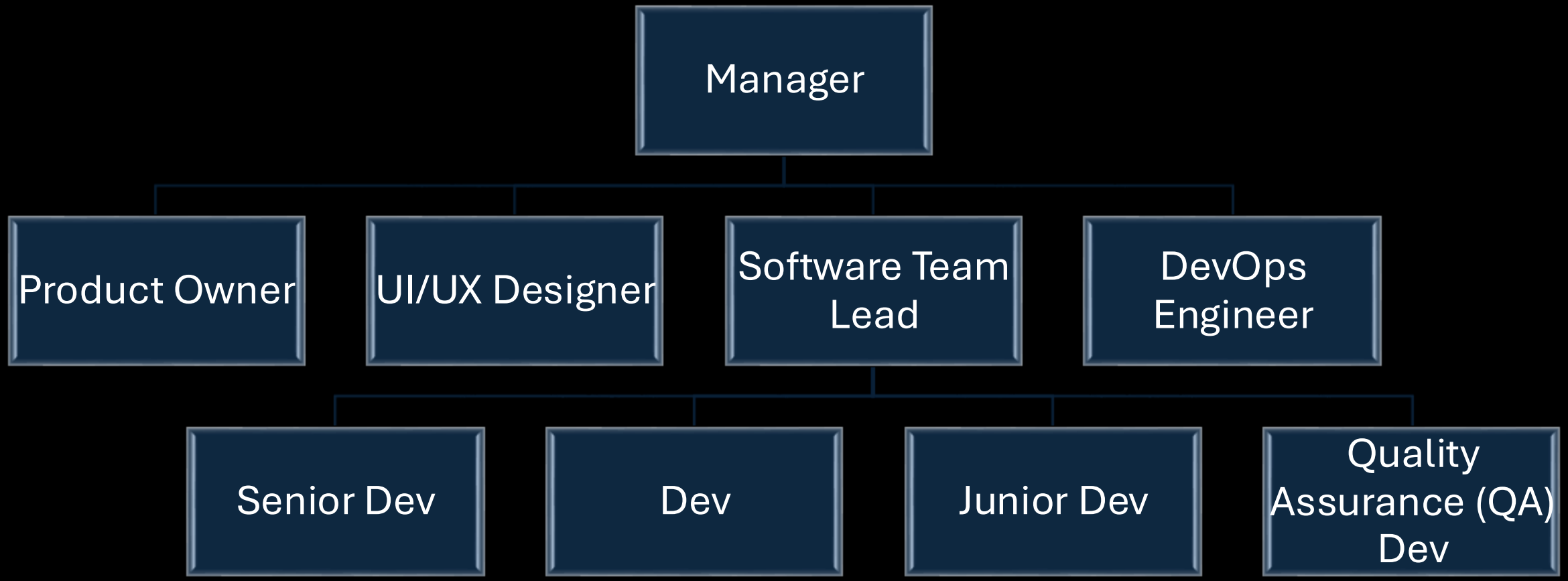
M5 – Ethics, Regex, and Presentations

Tools

- Java
- IntelliJ
- nciples and Patterns (things to do)
- Code Smells (things to avoid)
- An architecture
- APIs

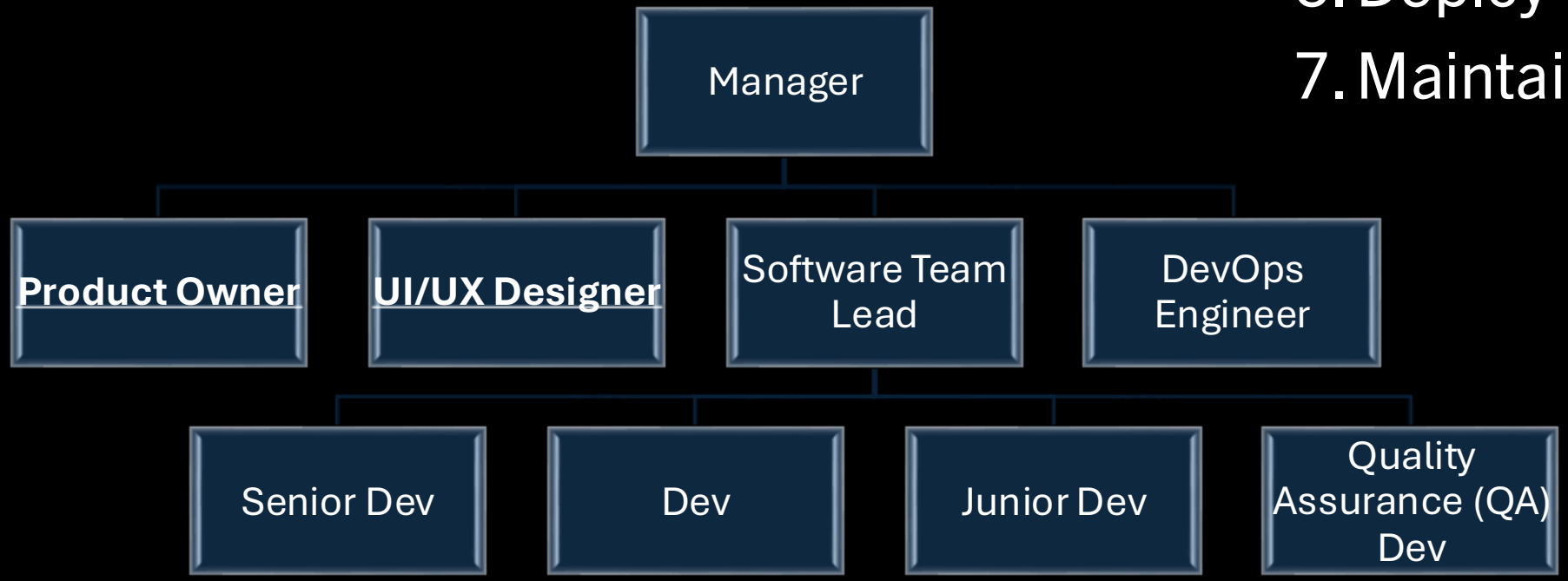
Going from OOP to OOD

Org chart of typical software roles



The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test
6. Deploy
7. Maintain

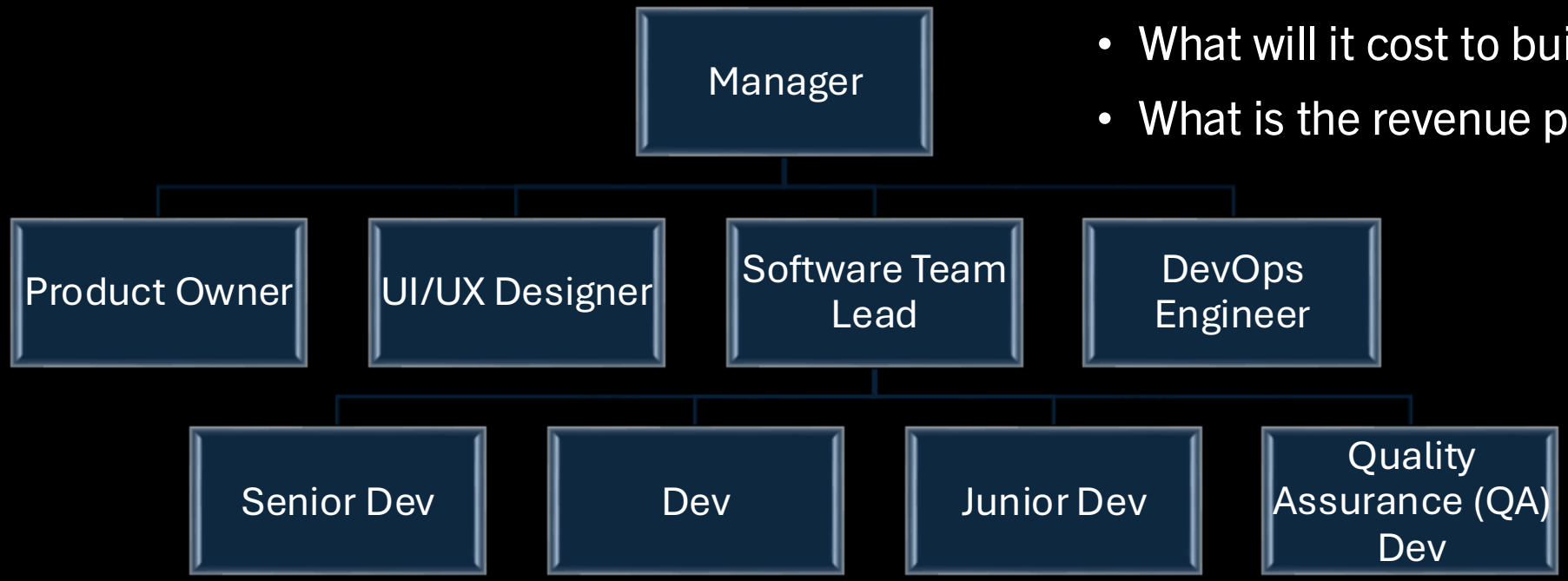


The software lifecycle

1. Planning and Analysis

- Product Owner, Client, Manager
- Goal: evaluate needs of end users, feasibility of project, revenue potential

- What does the client want built and what problems will it solve?
- What features will the end users need from the product?
- Roughly how long will it take to build?
- What will it cost to build it?
- What is the revenue potential?

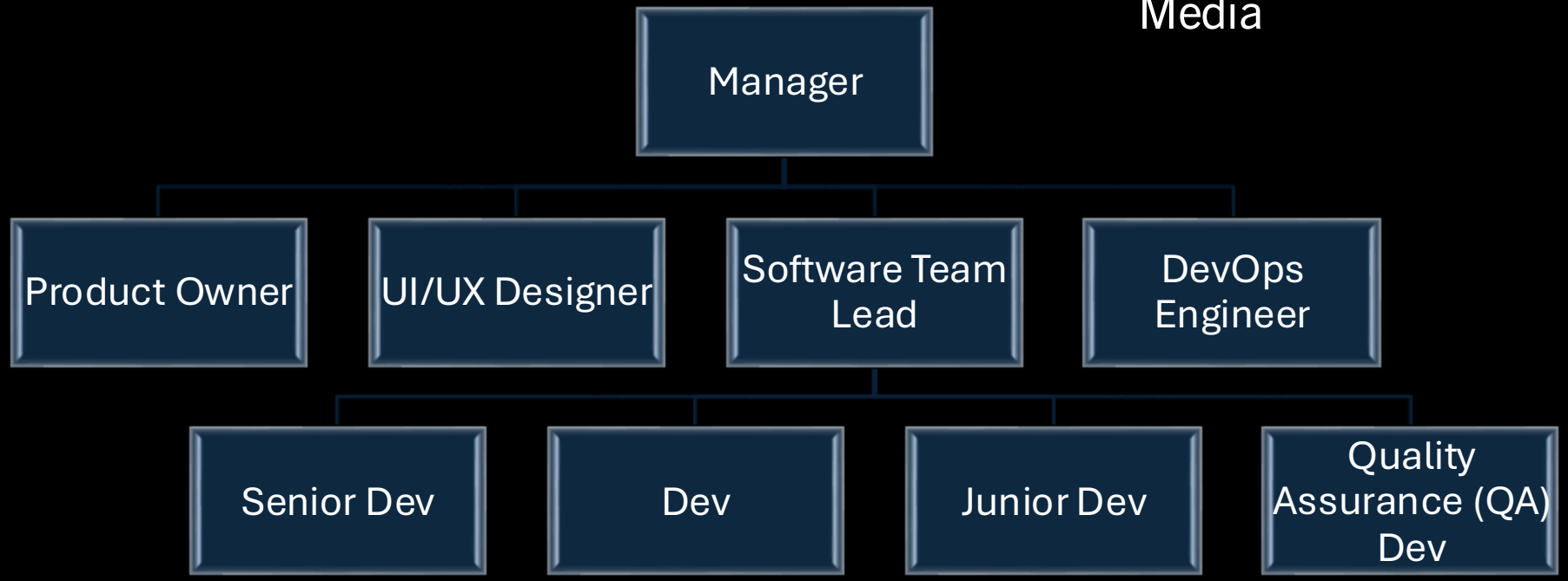


The software lifecycle

2. Define requirements

- Product Owner, Client, UI/UX Designer
- Goal: create clear requirements for the dev team

- Document the use cases for the software
- What will the end users want to do?
  - Make a list of specific tasks and acceptance criteria so you know when you're done
- CSC318 Design of Interactive Computational Media



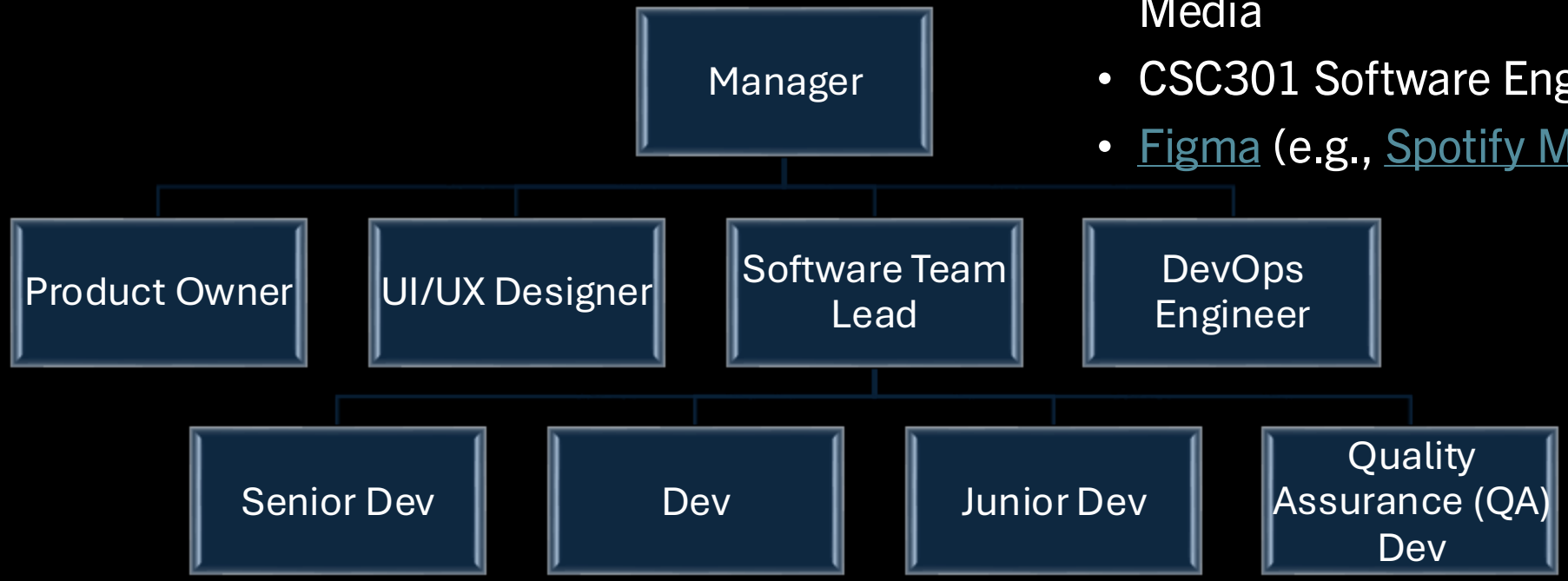
1. Planning and analysis
2. Define requirements

The software lifecycle

3. Design the UI and UX

- Product Owner, Client, UI/UX Designer
- Goal: develop a high-fidelity prototype

- No programming, just design
- Draw detailed pictures to capture what the screens will look like
- Validate prototype with customer
- CSC318 Design of Interactive Computational Media
- CSC301 Software Engineering
- Figma (e.g., Spotify Mobile UI Kit)



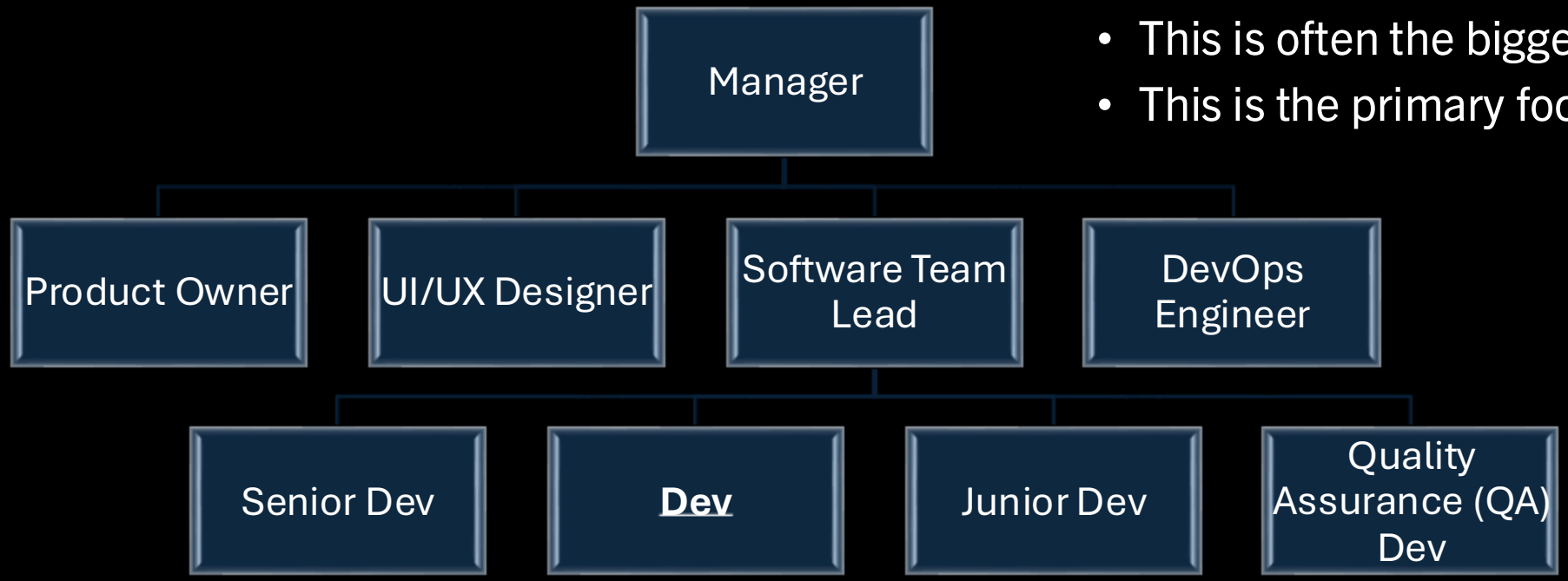
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX

4. Design and develop the code

- Software Team Lead, Devs, UI/UX Designer
- Goal: develop the actual product

- Design and grow a program that looks and behaves like the prototype
- Apply fancy techniques you'll learn in CSC207 to make it
  - Maintainable
  - Testable
- This is often the biggest part of the work
- This is the primary focus of CSC207



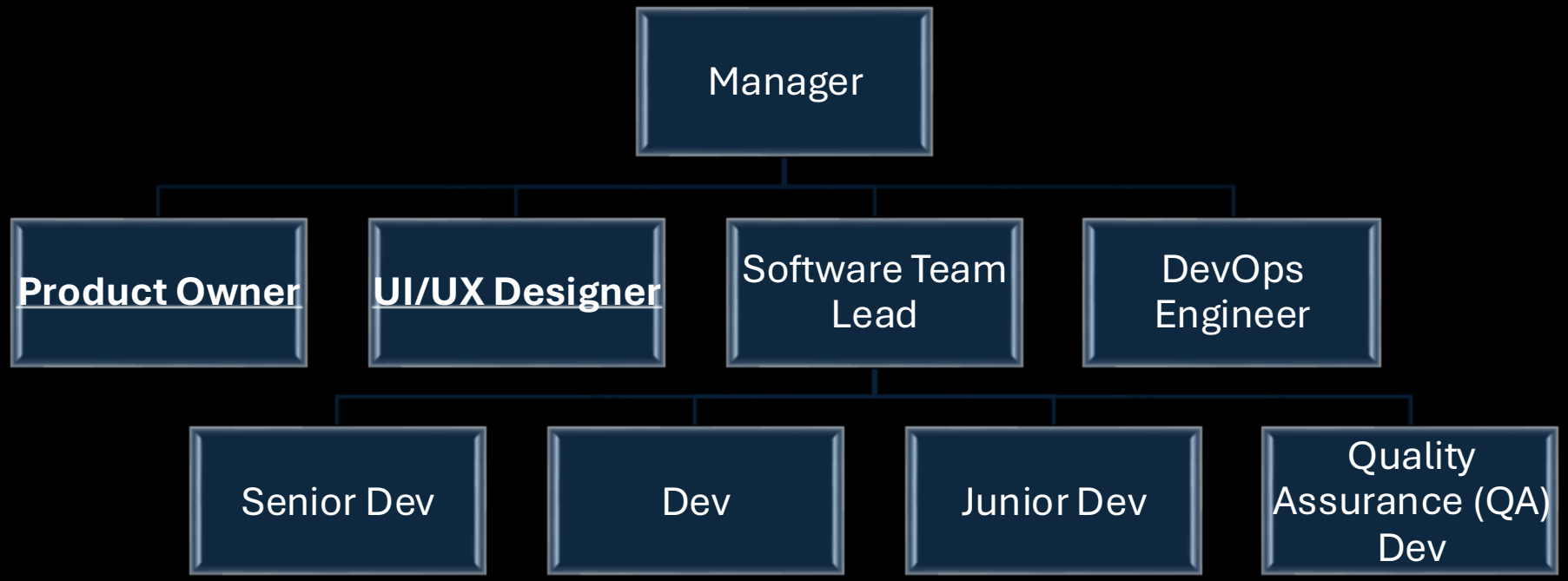
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code

5. Testing

- QA Dev, Product Owner, DevOps Engineer, Client, Devs
- Goal: make sure it works

- Types of testing:
  - performance, unit, integration, security, usability, acceptance
- CSC207 will focus on unit testing
- CSC301 Software Engineering



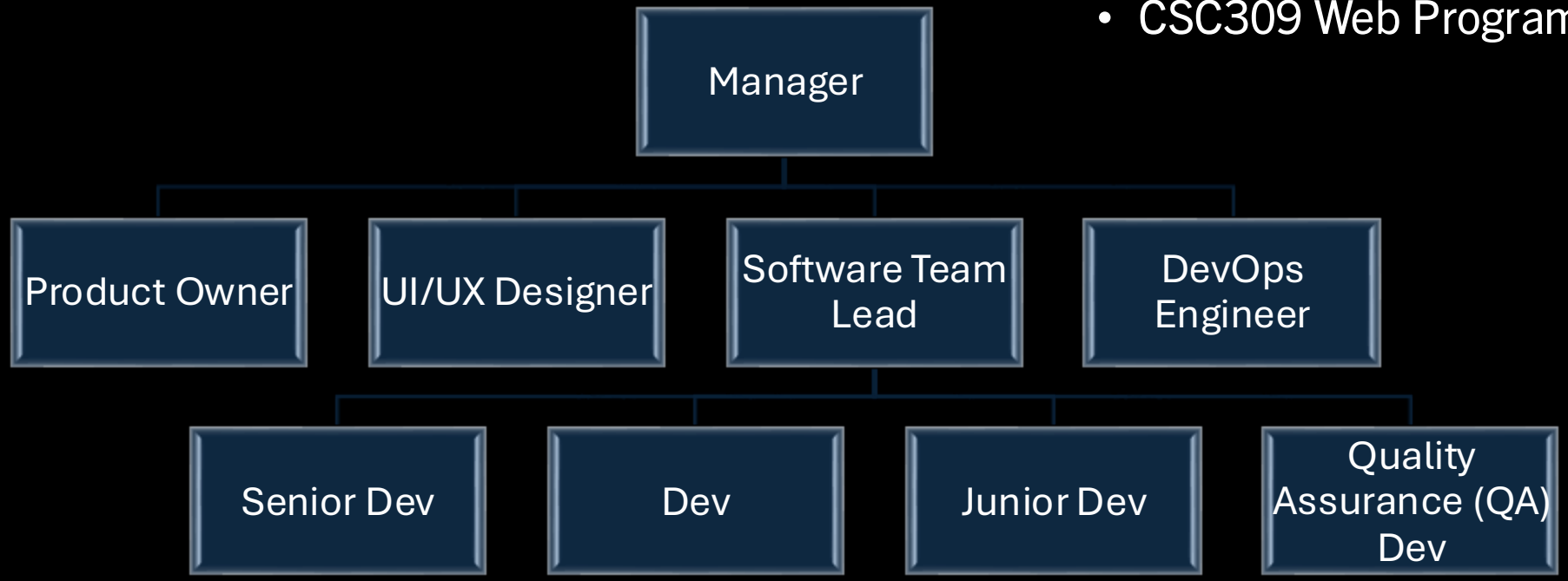
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test

6. Deployment

- DevOps Engineer, Devs, Product Owner, Client
- Goal: deliver it to end users

- Publish in app store
- Create and send an executable
- Update a website
- CSC301 Software Engineering
- CSC309 Web Programming



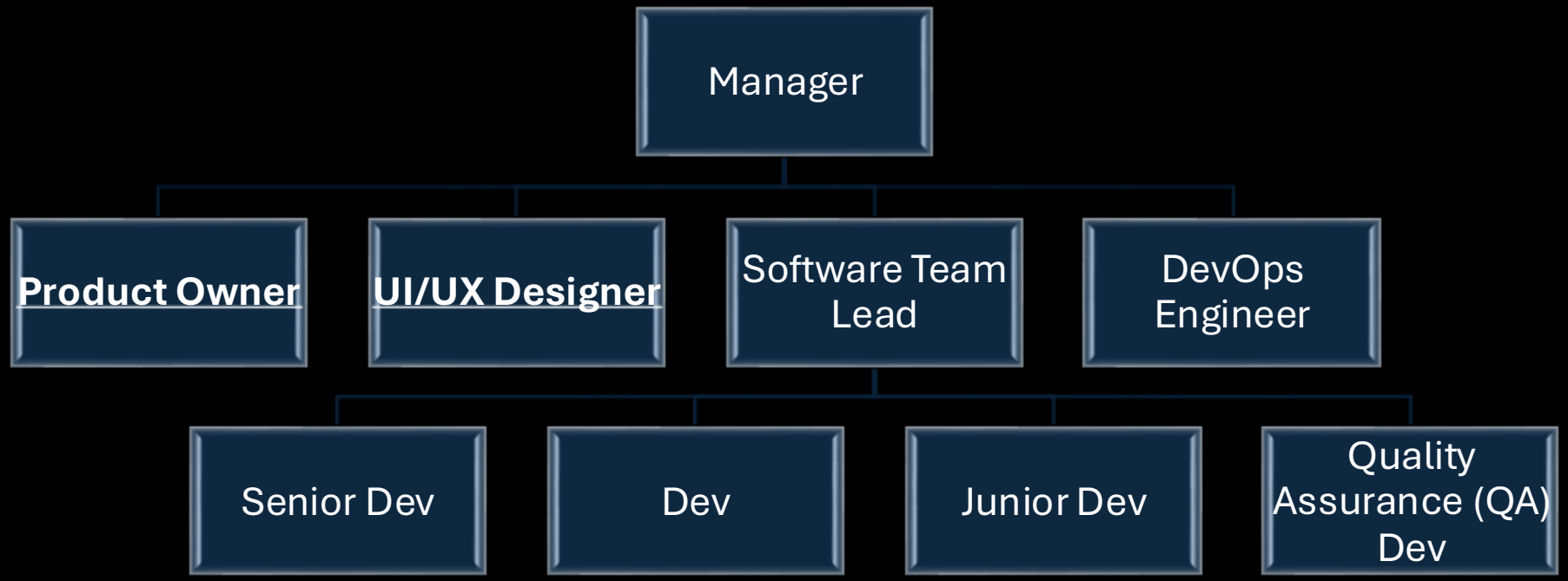
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test
6. Deploy

7. Maintenance

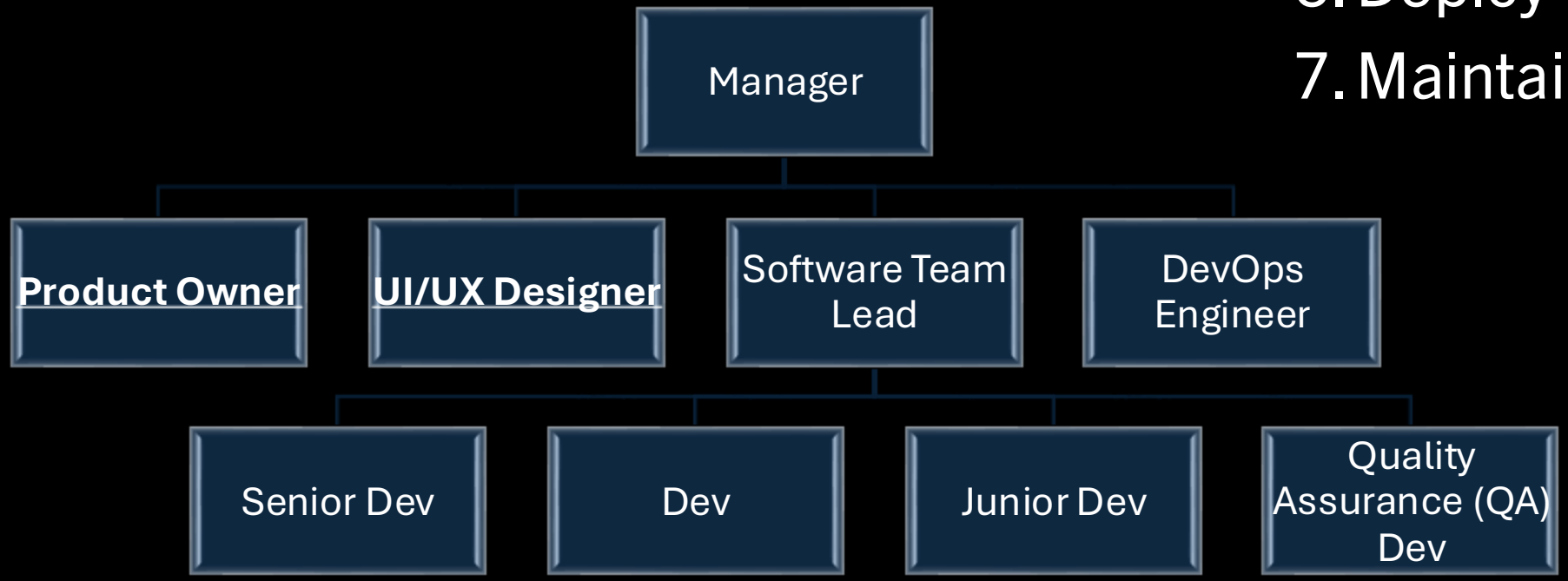
- DevOps Engineer, Devs, Product Owner, Client
- Goal: keep it going

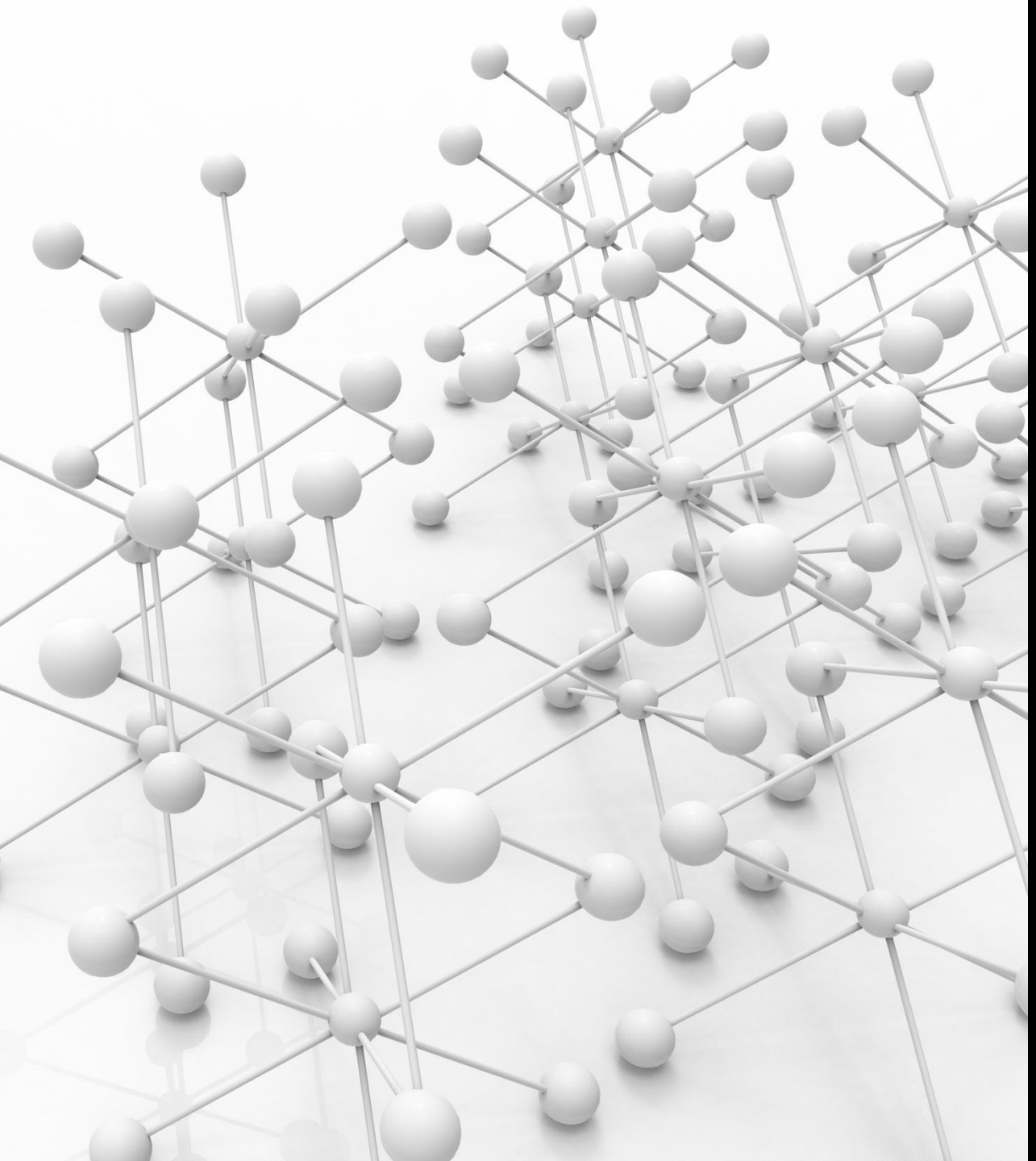
- Track and address bugs
- Ensure client needs are being met
- Plan for new features



The software lifecycle

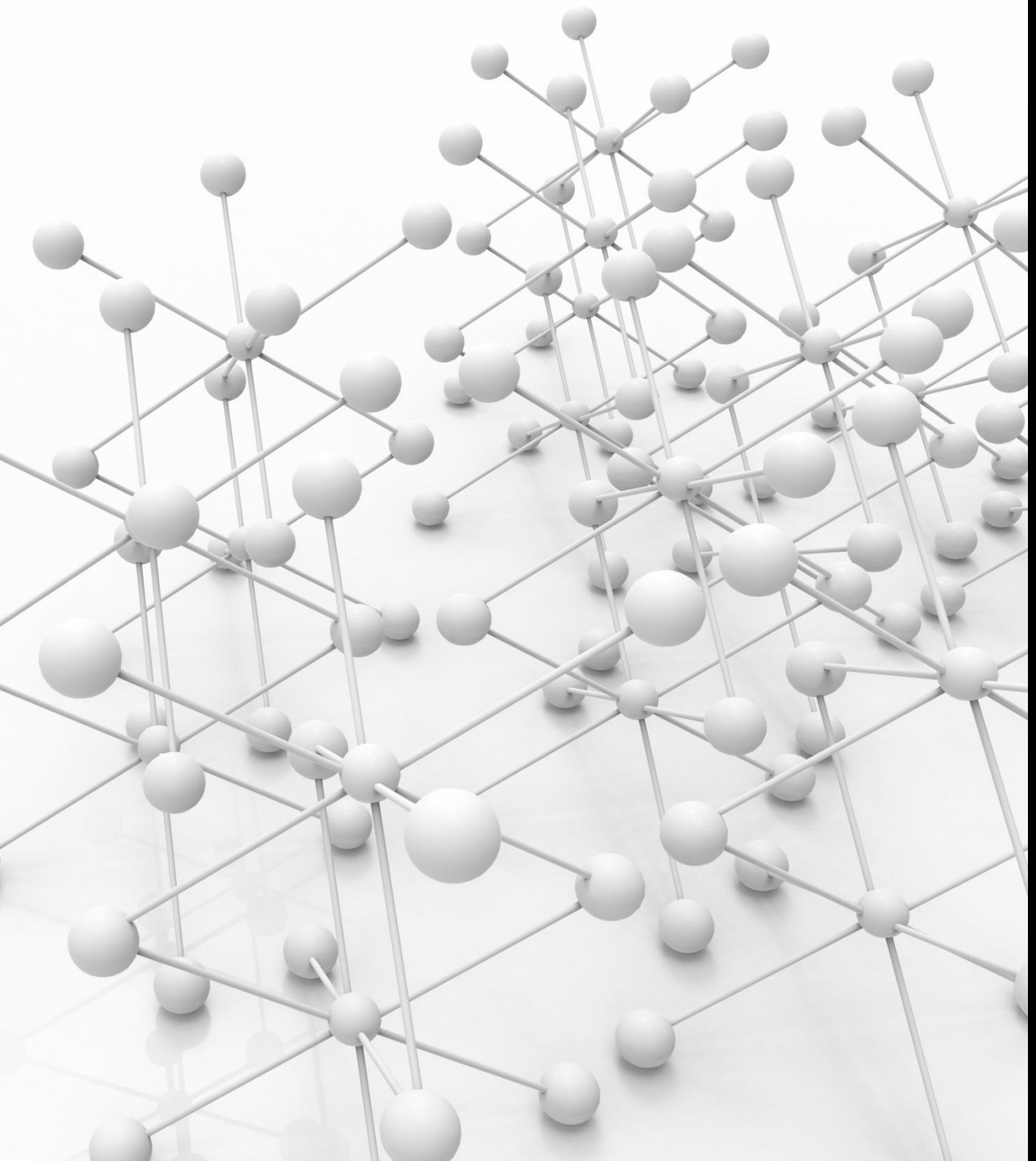
1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test
6. Deploy
7. Maintain





Design idea: a running program is a network of objects

- Interacting with the user interface causes a ripple of method calls through the graph
- Programmers conceptually organize this graph into layers: user interface, controller, model, database
- Design lore: keeping the layers cleanly separated is important for development, testing, and maintenance
- Lots of fancy design patterns to keep code pretty
- All aimed at “loose coupling, high cohesion”



Design idea: code for a use case interaction should be separate from all other code

Create a “use case interactor” object that knows how to handle a single user interaction

- Example: given two bank account numbers and an amount of money
  - Fetch the bank account objects from the database
  - Do the transfer
  - Save the info back to the database

Now the UI

- Has the user select two bank accounts
- Has the user choose how much to transfer
- Calls the use case interactor method to do the actual transfer

The Team-Based Learning Course Structure

TBL principles (Michaelsen & Richards 2005)

- Groups should be properly formed (e.g. Intellectual talent should be equally distributed among the groups). These teams are fixed for the whole course.
- Students are accountable for their pre-learning and for working in teams.
- Team assignments must promote both learning and team development.
- Students must receive frequent and immediate feedback.

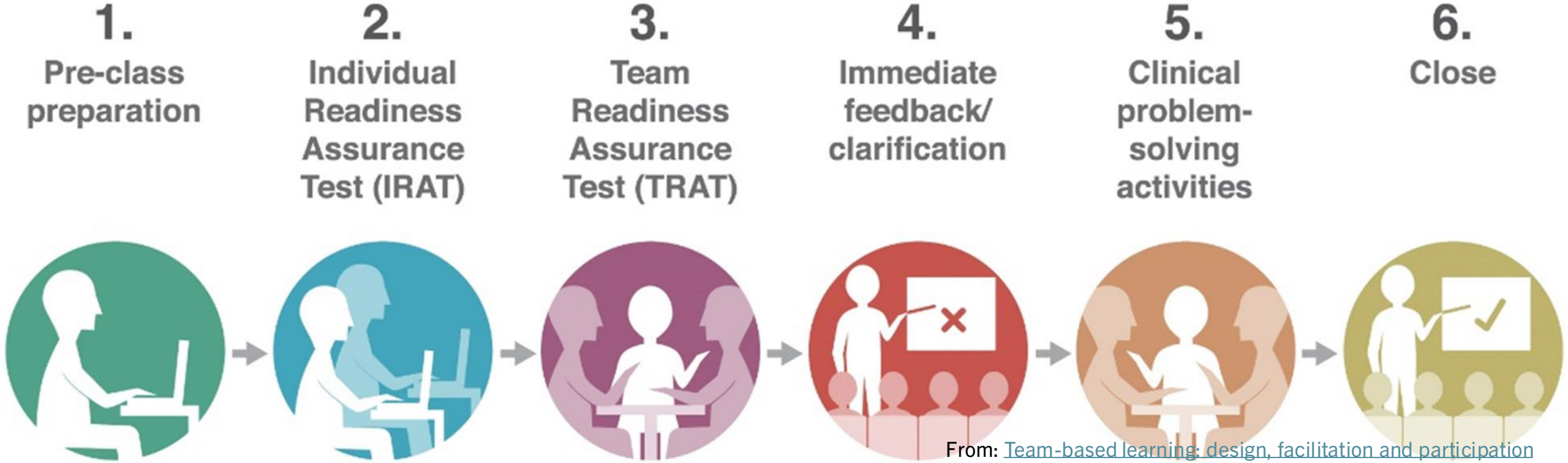
Course modules

Module 0: Java Fundamentals and Version Control
Module 1: Java OOP
Module 2: Design Principles
Module 3: Clean Architecture
Module 4: Design Patterns
Module 5: Ethics, Regex, and Presentations

2-week modules

- Team activities in each lecture
- Team activities in each lab
- Team project work outside of class time

- The iRAT and tRAT contain multiple choice questions on the prep material
- We'll practice the iRAT and tRAT in week 2



From: Team-based learning: design, facilitation and participation

Why Team Based Learning?

TBL encourages:

- Development of professionalism and communication skills
- Community building and sense of belonging (You all belong here!)

According to the literature on TBL, if you stay present and participate in class, you should expect:

To learn and remember more

To understand the material better

To master transferable skills that will help you in industry

Helping each other learn will benefit the team!

What do you want most in a teammate?

- What habits do you want your teammates to have?
- Are there any traits that you would want your ideal teammate to have?

What can you do to become a good teammate to the others on your team?

- Based on your previous answers, what are concrete things you can do to improve the way your team functions?
- What should you do if you don't understand something in an assignment?
- What should you do if someone else in the group is not participating?
- What should you do if you fall behind in the coursework?

Using the RULE to resolve conflicts

If you and your teammates disagree, consider using "RULE":

- Reflect – Reflect back to your teammate what you think their idea is, in case there has been a misunderstanding
- Understand – Ensure that they understand your idea too.
- Locate – Figure out exactly where your disagreement is. Possibly, you agree on most points and the disagreement is smaller than you think!
- Explore – Explore possible compromises or look at the pros and cons of your solution and the pros and cons of theirs.

TBL Demo

- How many foods can you think of that have carrots as an ingredient?
  - You have 2 minutes

TBL Demo part 2

Combine your list with a few people near you.

How many distinct foods did you and your team come up with?

Team formation survey

Link:
https://forms.office.com/r/iRcUw6SbX9

Hour 2: Learning Java and the course tools

- Our programming tools are *really* powerful
- Becoming an expert tool user can save you dozens, maybe hundreds, of programming hours over the next few years
- Rearranging, or *refactoring*, code is incredibly common
  - Make it work, then make it pretty
- Here's a quick tour

CSC108/148/110/111 stuff you know

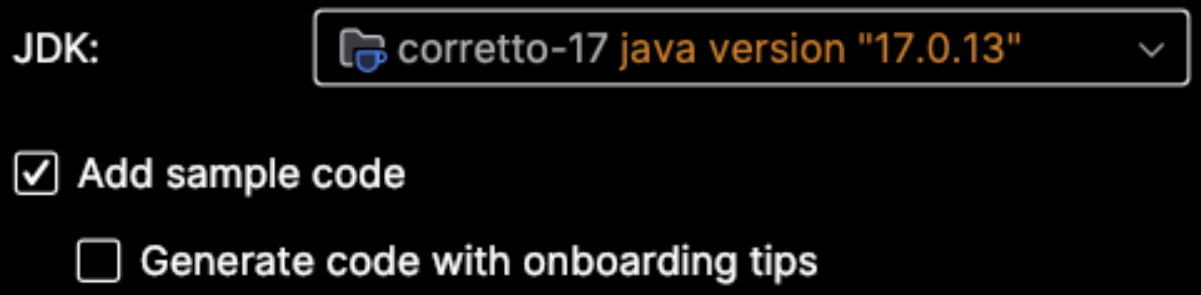
- value and type; expressions
- naming a value using an assignment statement (assigning a value to a variable)
- control flow: sequence of statements, if, while, for, function call, return statement, call stack, recursion
- ADTs and data structures: string, list, dictionary, linked list, stack, queue, tree
- classes and the objects they describe; composition; inheritance (OOP)
- some variables and methods are private (Python: use a leading _underscore)
- computational complexity (big-Oh)
- unit testing, debugging
- function and class design recipes — processes by which to write code

Creating a hello world program in IntelliJ

The Java ”Hello world!” program is surprisingly complex

IntelliJ:

- File —> New —> Project
- Select “Add sample code”
- Click button Create

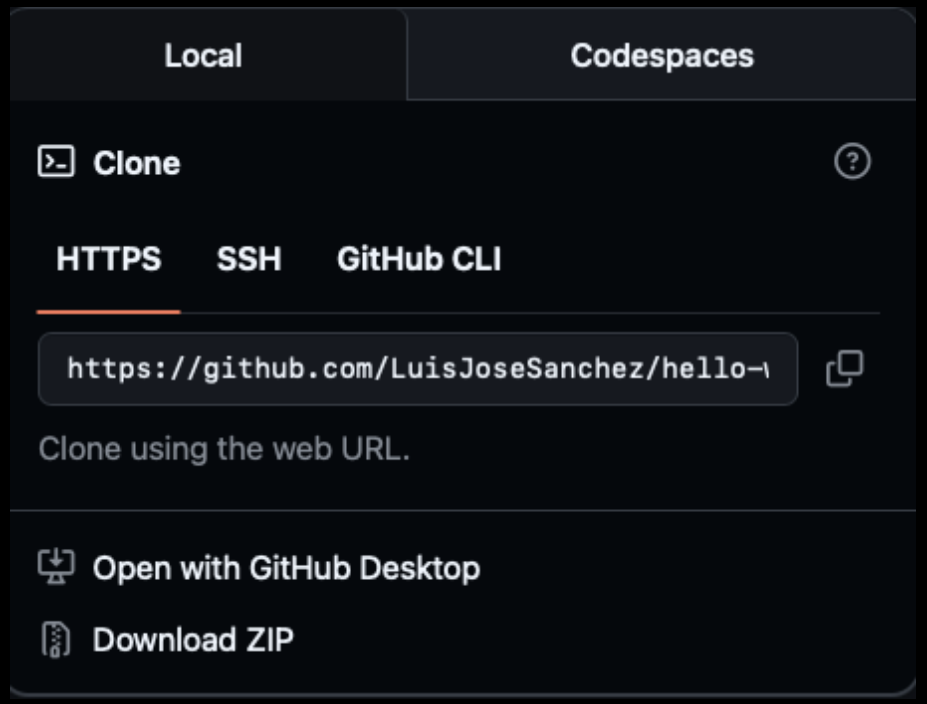



```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello world!");
    }
}
```

Cloning a repository in IntelliJ

You can download a program using Git

- Copy a URL from GitHub
  - https://github.com/LuisJoseSanchez/hello-world-java
- File —> New —> Project from Version Control . . .
- Paste the URL
- Click button Clone
- You'll need to choose a JDK to run it




```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello world!");
  }
}
```

Exploring a Java program using IntelliJ

- https://github.com/paulgries/UniversityExample